Vulnerability Assessment and Penetration Testing (VAPT)



Vyankatesh Andil

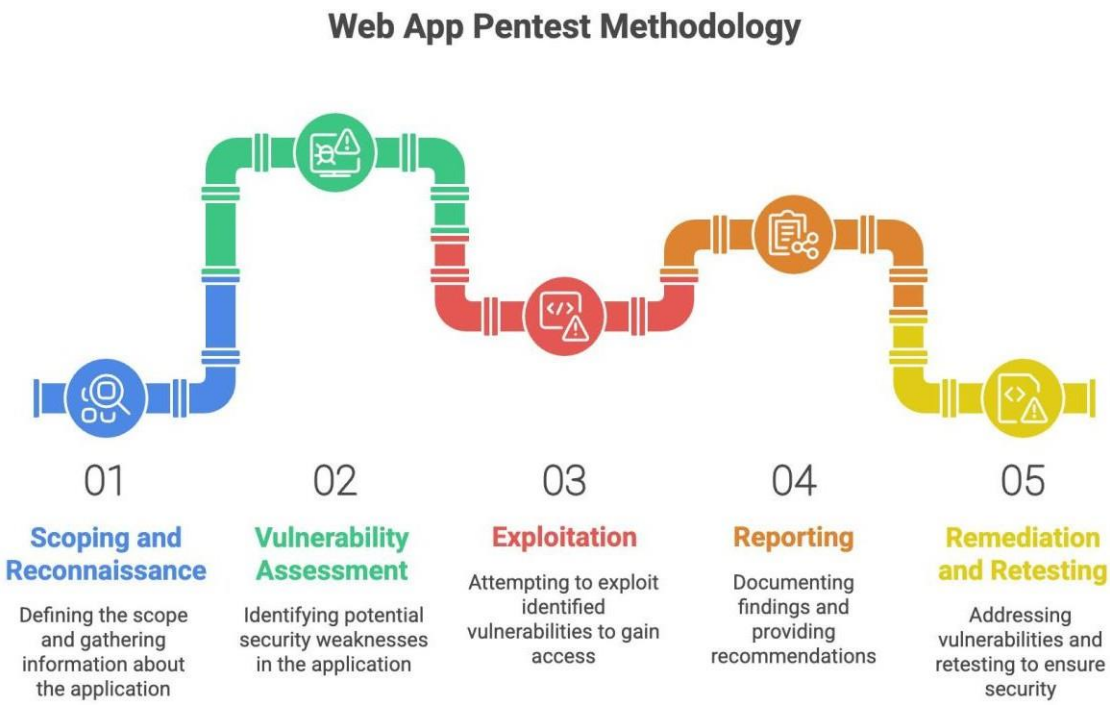
Sr. No.	Title	Page No.
1	Title Page	1
2	Table of Contents	2
3	Introduction	3
4	Objectives	4
5	Scope	4
6	Methodology	5
7	Phase 1: Planning and Authorization	7
8	Phase 2: Information Gathering (Reconnaissance)	7
9	Phase 3: Website Enumeration	10
10	Phase 4: Fingerprinting	11
11	Phase 5: Vulnerability Assessment	14
12	Nessus & Acunetix	16
13	Phase 6: Manual Penetration Testing	19
14	6.1 Authentication Testing	19
15	6.2 SQL Injection Testing	20
16	Manual Exploitation & Burp Suite Evidence	21
17	Cross-Site Scripting (XSS) Testing	24
18	Network Traffic Analysis (Wireshark)	26
19	6.3 SSL/TLS Testing	29
20	Phase 7: Exploitation	30
21	Phase 8: Risk Assessment	30
22	Phase 9: Remediation	31
23	Tools Used	32
24	Conclusion	33

VULNERABILITY ASSESSMENT AND PENETRATION TESTING (VAPT) ON A WEB APPLICATION

1. Introduction

Web applications have become an essential part of modern organizations for providing online services such as e-commerce, banking, education, healthcare, and social networking. Since these applications store sensitive user information, they are common targets for cyberattacks. Vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, and Insecure Direct Object References (IDOR) can allow attackers to steal data, modify application functionality, or gain unauthorized access.

Vulnerability Assessment and Penetration Testing (VAPT) is a comprehensive security assessment methodology used to identify, verify, and exploit security weaknesses in a web application under authorized conditions. Vulnerability Assessment identifies potential security flaws, while Penetration Testing validates whether those vulnerabilities can be successfully exploited by an attacker.



2. Objectives

The objectives of this project are:

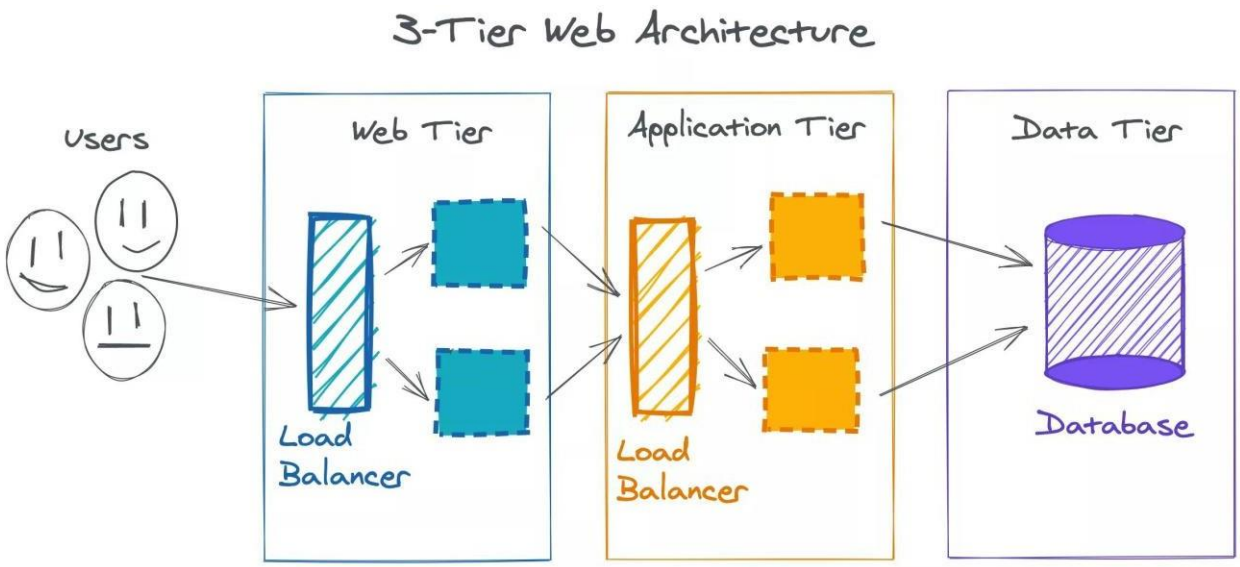
- Identify security vulnerabilities in the web application.
- Analyze the impact of discovered vulnerabilities.
- Demonstrate the exploitation of vulnerabilities in a controlled environment.
- Provide recommendations to mitigate identified security risks.
- Improve the overall security of the application.

3. Scope

The assessment includes the following components:

- Login and Authentication System
- User Registration
- User Dashboard
- Search Functionality
- File Upload Module
- Contact Form
- Session Management
- Database Interaction
- HTTP Headers
- Server Configuration

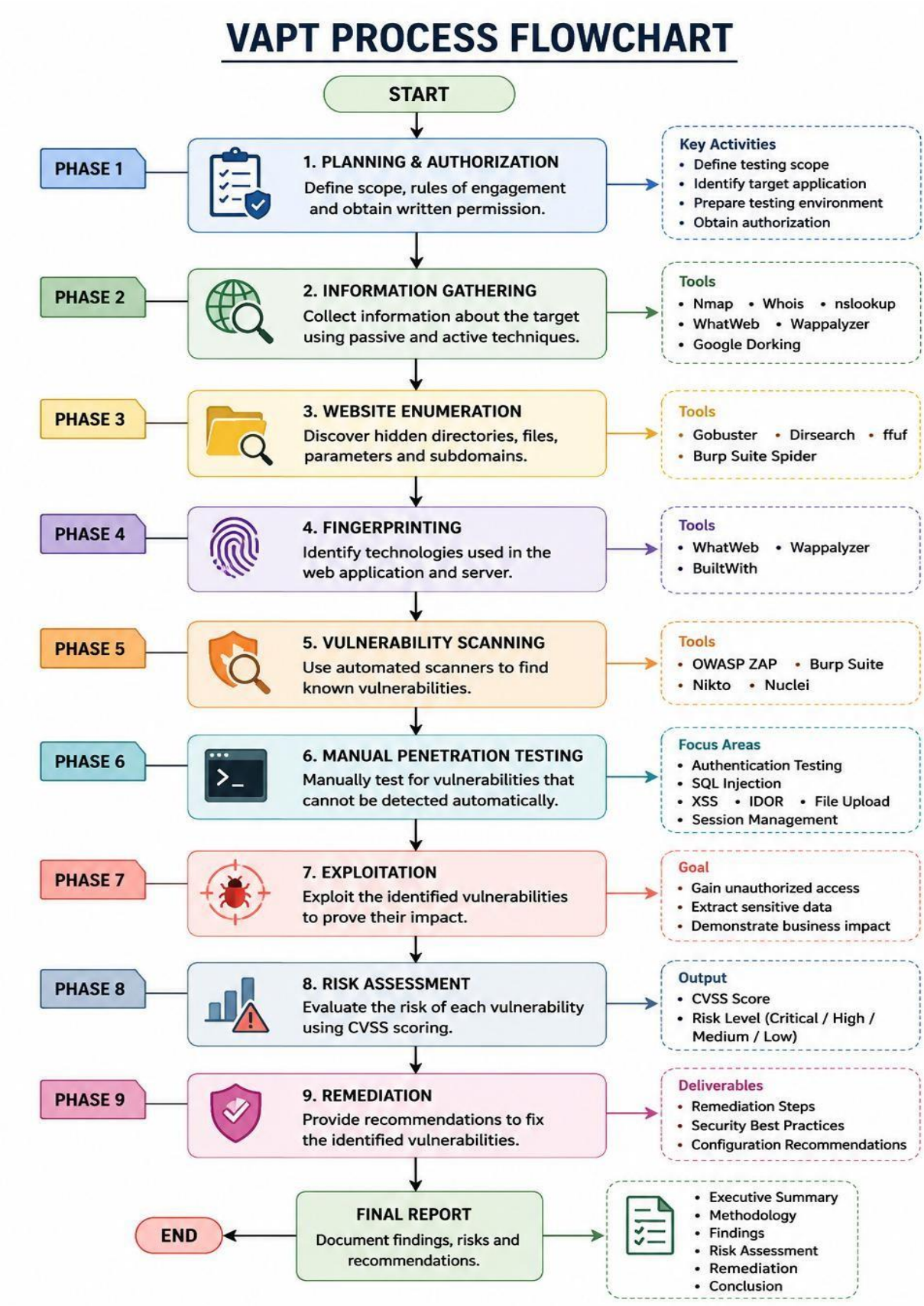
The assessment excludes denial-of-service attacks and testing of third-party services not owned by the organization.



4. Methodology

The VAPT process follows the following phases:

1. Planning and Authorization
2. Information Gathering
3. Reconnaissance
4. Vulnerability Scanning
5. Manual Testing
6. Exploitation
7. Risk Analysis
8. Reporting
9. Remediation


VAPT PROCESS FLOWCHART
START
PHASE 1
1. PLANNING & AUTHORIZATION
Define scope, rules of engagement and obtain written permission.
Key Activities
• Define testing scope
• Identify target application
• Prepare testing environment
• Obtain authorization
PHASE 2
2. INFORMATION GATHERING
Collect information about the target using passive and active techniques.
Tools
• Nmap • Whois • nslookup
• WhatWeb • Wappalyzer
• Google Dorking
PHASE 3
3. WEBSITE ENUMERATION
Discover hidden directories, files, parameters and subdomains.
Tools
• Gobuster • Dirsearch • ffuf
• Burp Suite Spider
PHASE 4
4. FINGERPRINTING
Identify technologies used in the web application and server.
Tools
• WhatWeb • Wappalyzer
• BuiltWith
PHASE 5
5. VULNERABILITY SCANNING
Use automated scanners to find known vulnerabilities.
Tools
• OWASP ZAP • Burp Suite
• Nikto • Nuclei
PHASE 6
6. MANUAL PENETRATION TESTING
Manually test for vulnerabilities that cannot be detected automatically.
Focus Areas
• Authentication Testing
• SQL Injection
• XSS • IDOR • File Upload
• Session Management
PHASE 7
7. EXPLOITATION
Exploit the identified vulnerabilities to prove their impact.
Goal
• Gain unauthorized access
• Extract sensitive data
• Demonstrate business impact
PHASE 8
8. RISK ASSESSMENT
Evaluate the risk of each vulnerability using CVSS scoring.
Output
• CVSS Score
• Risk Level (Critical / High / Medium / Low)
PHASE 9
9. REMEDIATION
Provide recommendations to fix the identified vulnerabilities.
Deliverables
• Remediation Steps
• Security Best Practices
• Configuration Recommendations
FINAL REPORT
Document findings, risks and recommendations.
END
• Executive Summary
• Methodology
• Findings
• Risk Assessment
• Remediation
• Conclusion

Phase 1: Planning and Authorization

Before starting the assessment, proper authorization must be obtained from the application owner. Testing without permission is illegal and unethical.

Activities Performed

- Define testing scope.
- Identify target URL/IP.
- Prepare testing environment.
- Install required tools.
- Obtain written permission.

Deliverables

- Scope Document
- Authorization Letter
- Testing Plan

Phase 2: Information Gathering (Reconnaissance)

Information gathering is the first technical phase of VAPT. The purpose is to collect as much information as possible about the target application.

Information Collected

- Target IP Address
- Domain Name
- DNS Records
- Server Technology
- Programming Language
- Framework

- Database
- Open Ports
- Running Services

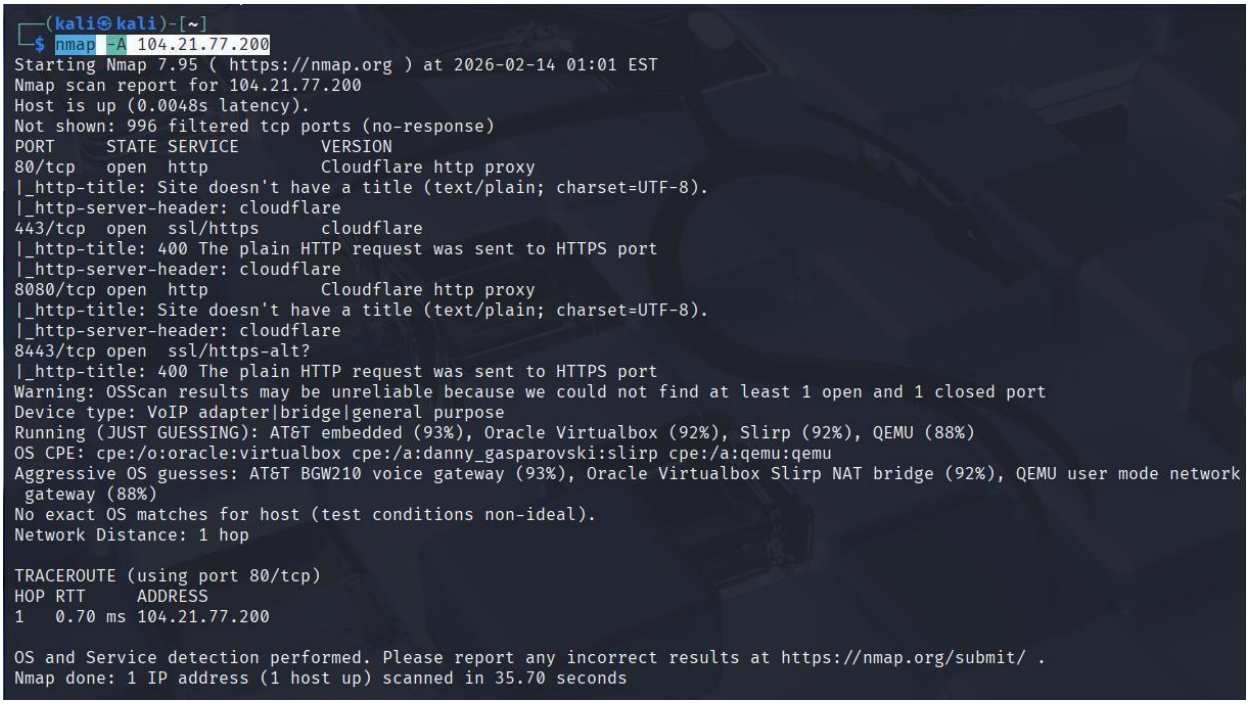
Tools Used

- Nmap
- Whois
- nslookup
- WhatWeb
- Wappalyzer

Commands

nmap -A target-ip

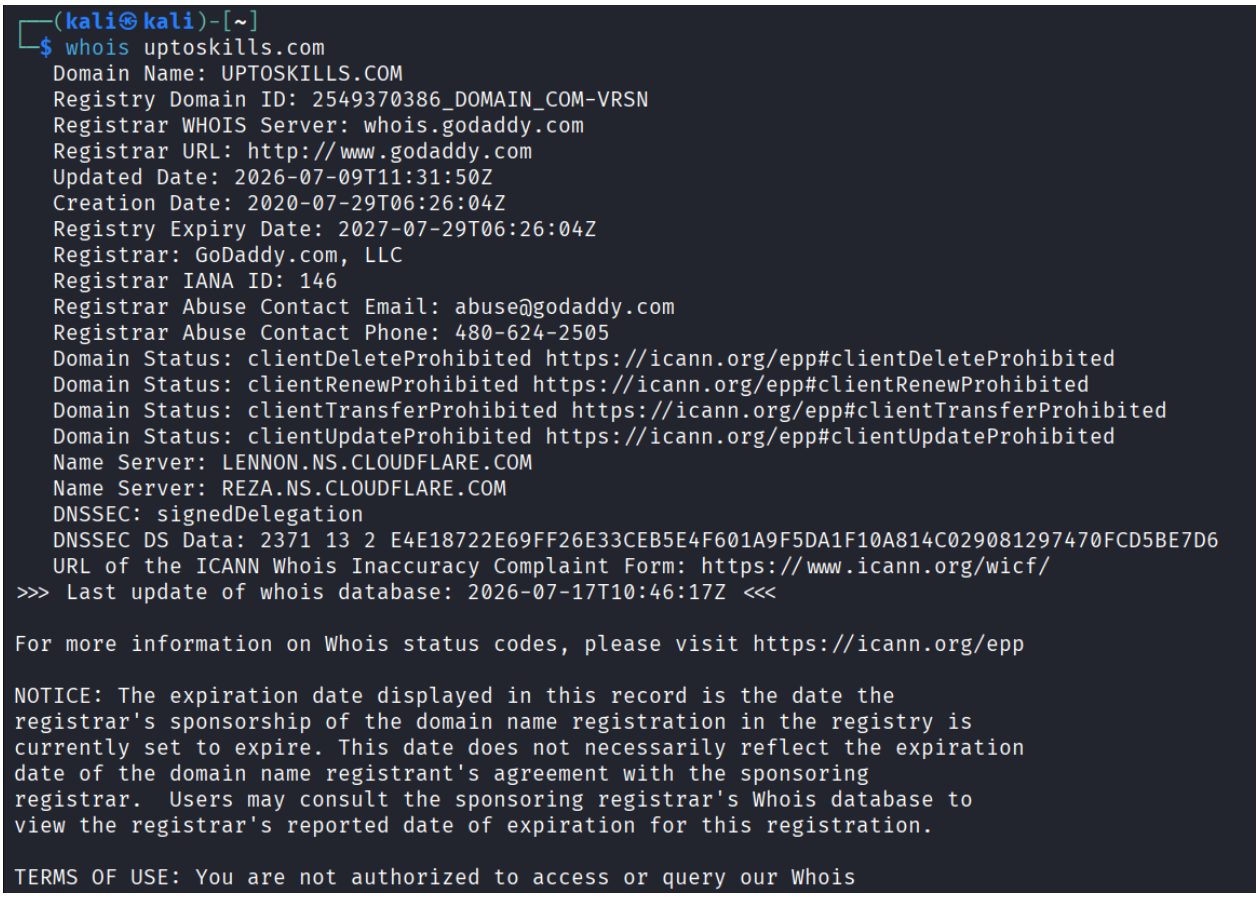
Aim: An aggressive scan (-A) collects detailed information by enabling OS detection, service/version detection, scripts, and traceroute in a single command.



whatweb http://target-url



whois target-domain



Expected Output

- Apache Server
- PHP 8.x
- MySQL Database

- Port 80 Open
- Port 443 Open

Phase 3: Website Enumeration

Enumeration identifies hidden resources inside the application.

Objectives

- Discover hidden directories.
- Find backup files.
- Identify admin panels.
- Detect exposed configuration files.

Tools

- Gobuster
- Dirsearch
- ffuf

Example Command

```
gobuster dir -u http://target.com -w /usr/share/wordlists/dirb/common.txt
```

Aim: Find hidden directories and files on a website.



Findings

The scan may reveal:

- /admin
- /login
- /uploads
- /backup
- /config

Phase 4: Fingerprinting

Fingerprinting identifies technologies used to build the application.

Information Identified

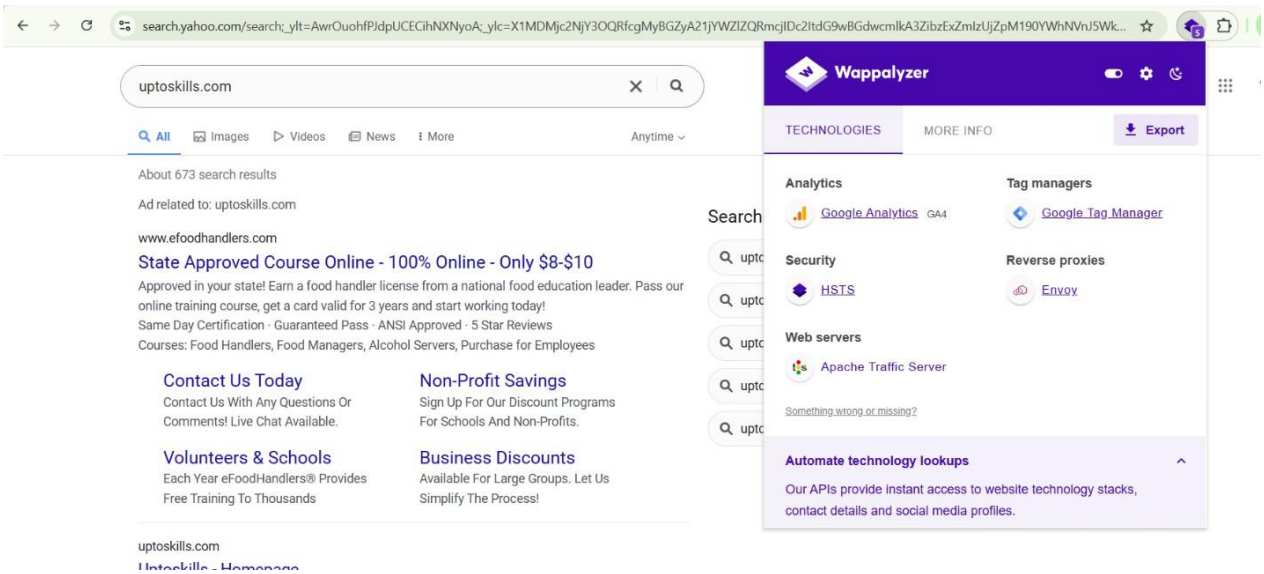
- Web Server
- Programming Language
- CMS
- Framework
- Database
- Operating System

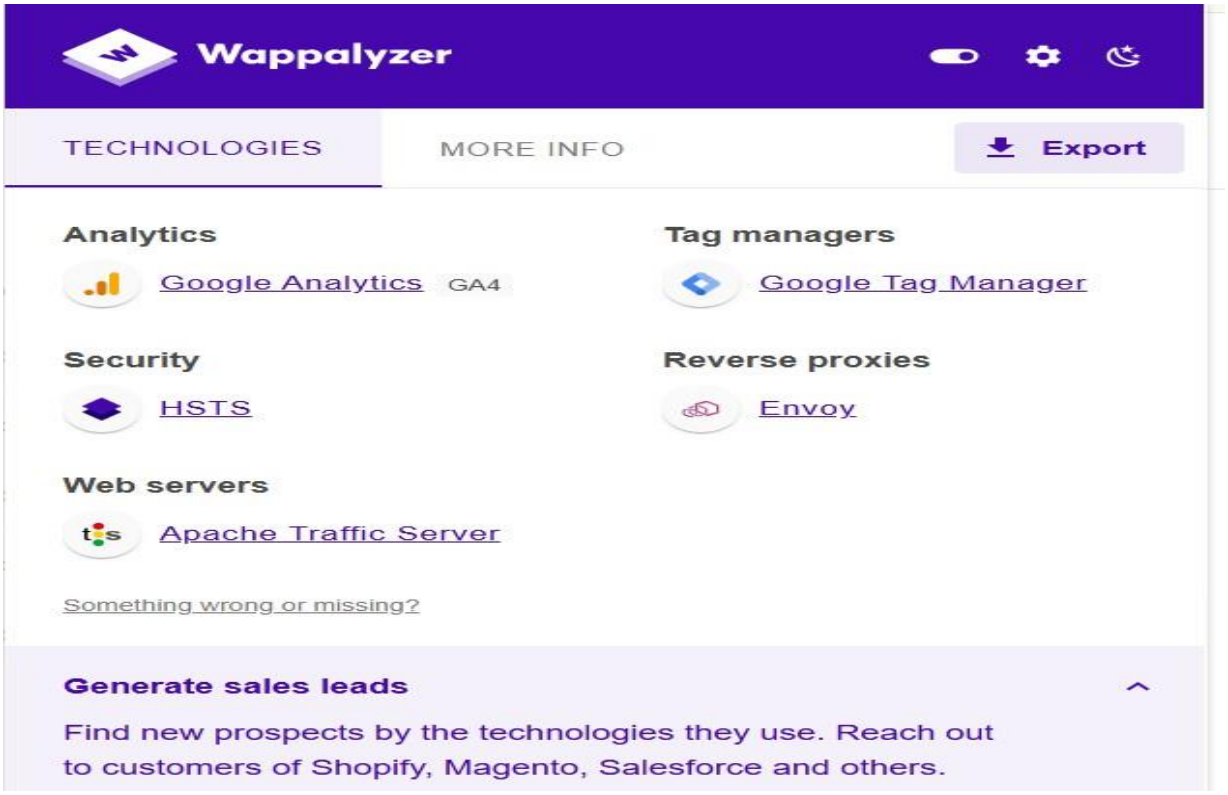
Tools

- **WhatWeb**

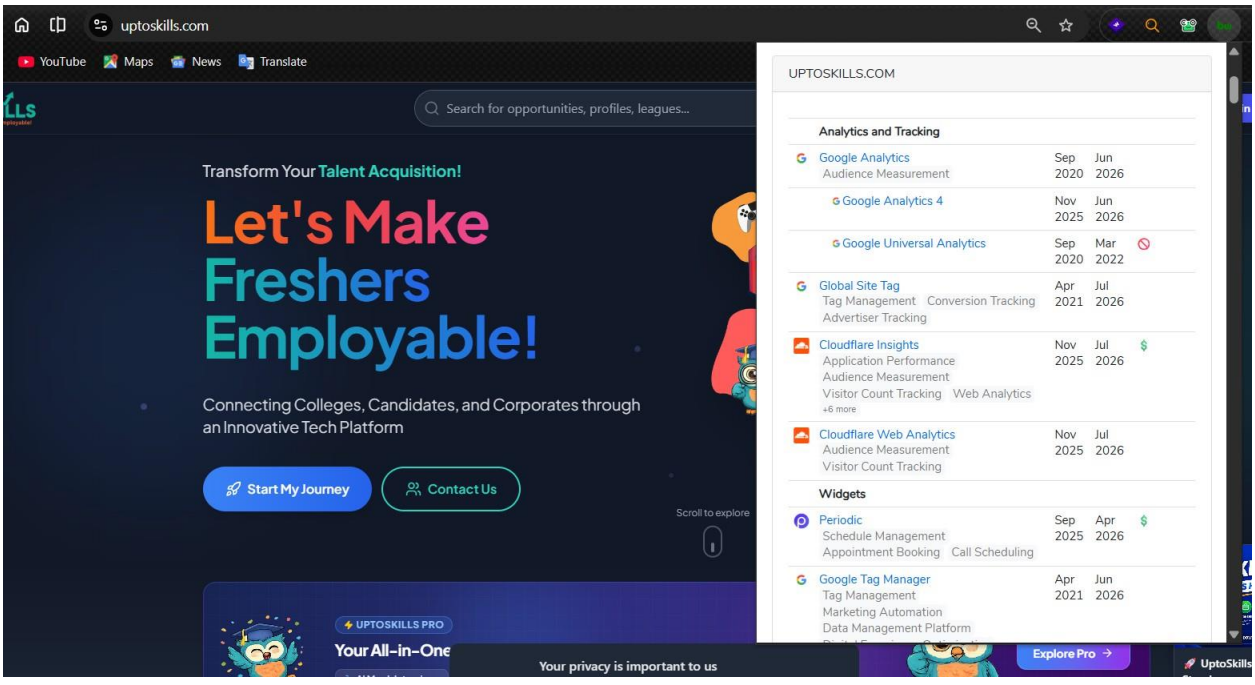
```
┌──(kali㉿kali)-[~]
└─$ whatweb uptoskills.com
http://uptoskills.com [301 Moved Permanently] Country[UNITED STATES][US], HTTPServer[cloudflare], IP[104.21.77.200], RedirectLocation[https://uptoskills.com/
 UncommonHeaders[speculation-rules,report-to,nel,cf-ray,alt-svc]
https://uptoskills.com [200 OK] Country[UNITED STATES][US], Email[info@uptoskills.com], HTML5, HTTPServer[cloudflare], IP[104.21.77.200], Meta-Author[Uptoski
2090,IN,India,New Delhi], Open-Graph-Protocol[website], Script[application/ld+json,module], Strict-Transport-Security[max-age=31536000; includeSubDomains; pr
s AI-Powered Hiring & Career Ecosystem], UncommonHeaders[report-to,nel,cf-cache-status,access-control-allow-origin,x-permitted-cross-domain-policies,link,con
n-opener-policy,cross-origin-resource-policy,permissions-policy,referrer-policy,x-content-type-options,x-download-options,speculation-rules,cf-ray,alt-svc],
tection[1; mode=block]
https://uptoskills.com/ [200 OK] Country[RESERVED][ZZ], Email[info@uptoskills.com], HTML5, HTTPServer[cloudflare], IP[172.67.211.113], Meta-Author[Uptoskills
0,IN,India,New Delhi], Open-Graph-Protocol[website], Script[application/ld+json,module], Strict-Transport-Security[max-age=31536000; includeSubDomains; prelo
I-Powered Hiring & Career Ecosystem], UncommonHeaders[report-to,nel,cf-cache-status,access-control-allow-origin,x-permitted-cross-domain-policies,link,conten
pener-policy,cross-origin-resource-policy,permissions-policy,referrer-policy,x-content-type-options,x-download-options,speculation-rules,cf-ray,alt-svc], X-F
tion[1; mode=block]
```

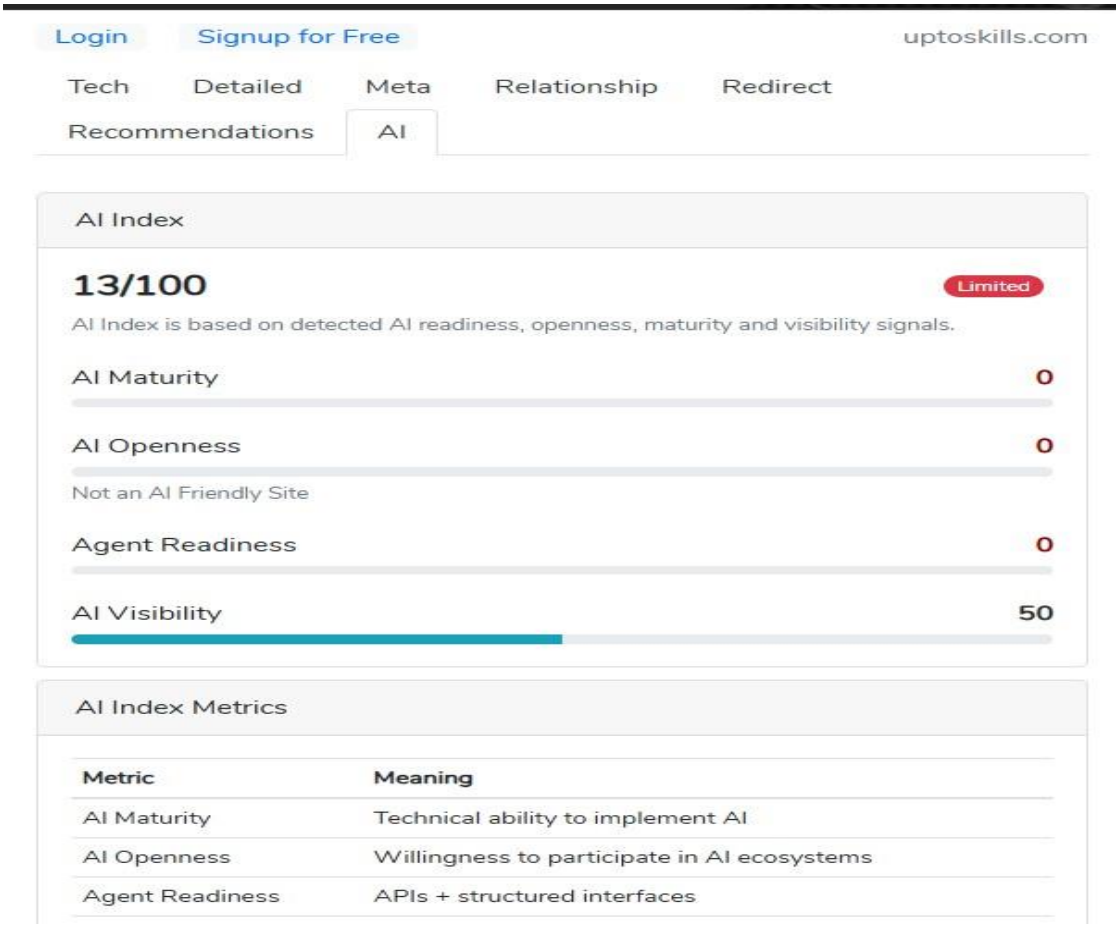
- **Wappalyzer**




Wappalyzer
TECHNOLOGIES
MORE INFO
Export
Analytics
Google Analytics GA4
Tag managers
Google Tag Manager
Security
HSTS
Reverse proxies
Envoy
Web servers
Apache Traffic Server
Something wrong or missing?
Generate sales leads
Find new prospects by the technologies they use. Reach out to customers of Shopify, Magento, Salesforce and others.

- **BuiltWith**


uptoskills.com
YouTube
Maps
News
Translate
Search for opportunities, profiles, leagues...
Transform Your Talent Acquisition!
Let's Make Freshers Employable!
Connecting Colleges, Candidates, and Corporates through an Innovative Tech Platform
Start My Journey
Contact Us
Scroll to explore
UPTOSKILLS.COM
Analytics and Tracking
Google Analytics
Audience Measurement
Sep 2020 Jun 2026
Google Analytics 4
Nov 2025 Jun 2026
Google Universal Analytics
Sep 2020 Mar 2022
Global Site Tag
Tag Management Conversion Tracking
Advertiser Tracking
Apr 2021 Jul 2026
Cloudflare Insights
Application Performance
Audience Measurement
Visitor Count Tracking Web Analytics
+6 more
Nov 2025 Jul 2026
Cloudflare Web Analytics
Audience Measurement
Visitor Count Tracking
Nov 2025 Jul 2026
Widgets
Periodic
Schedule Management
Appointment Booking Call Scheduling
Sep 2025 Apr 2026
Google Tag Manager
Tag Management
Marketing Automation
Data Management Platform
Apr 2021 Jun 2026
UPTOSKILLS PRO
Your All-in-One
Your privacy is important to us
Explore Pro

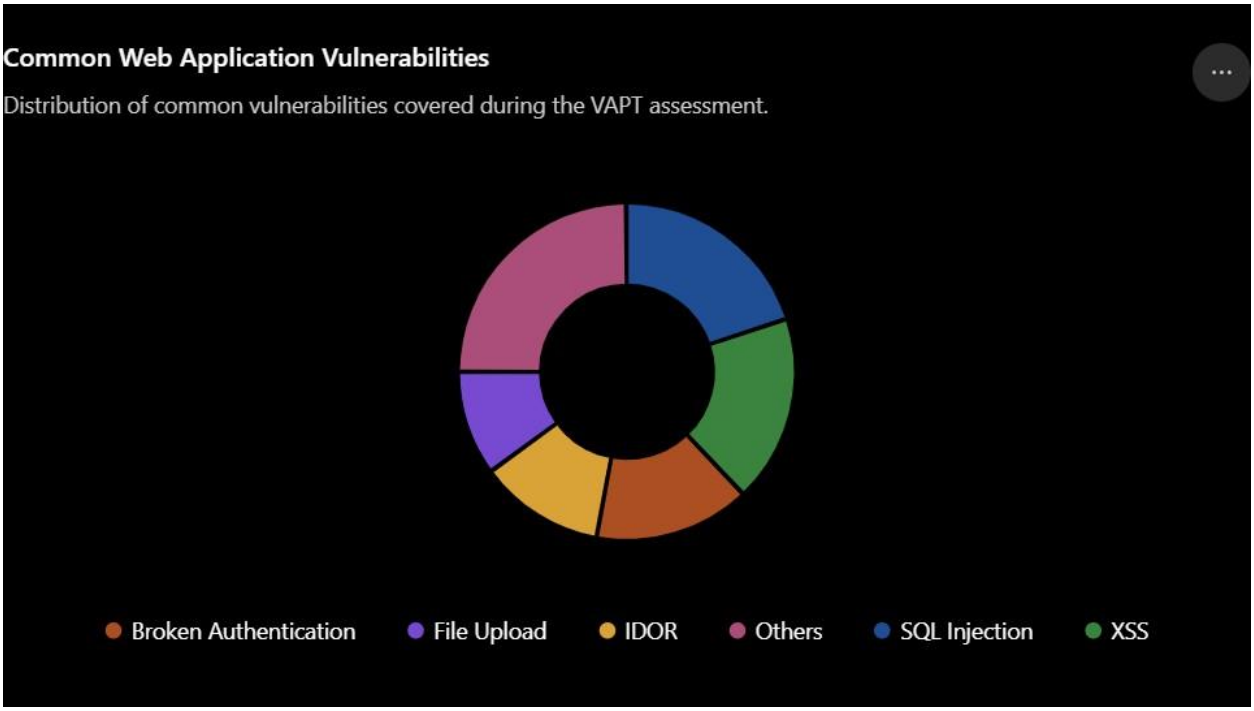


Example Output

- Apache 2.4
- PHP
- Bootstrap
- MySQL
- Linux Ubuntu

Phase 5: Vulnerability Assessment

Automated scanners identify known vulnerabilities.



Tools Used

- OWASP ZAP
- Burp Suite
- Nikto
- Nuclei

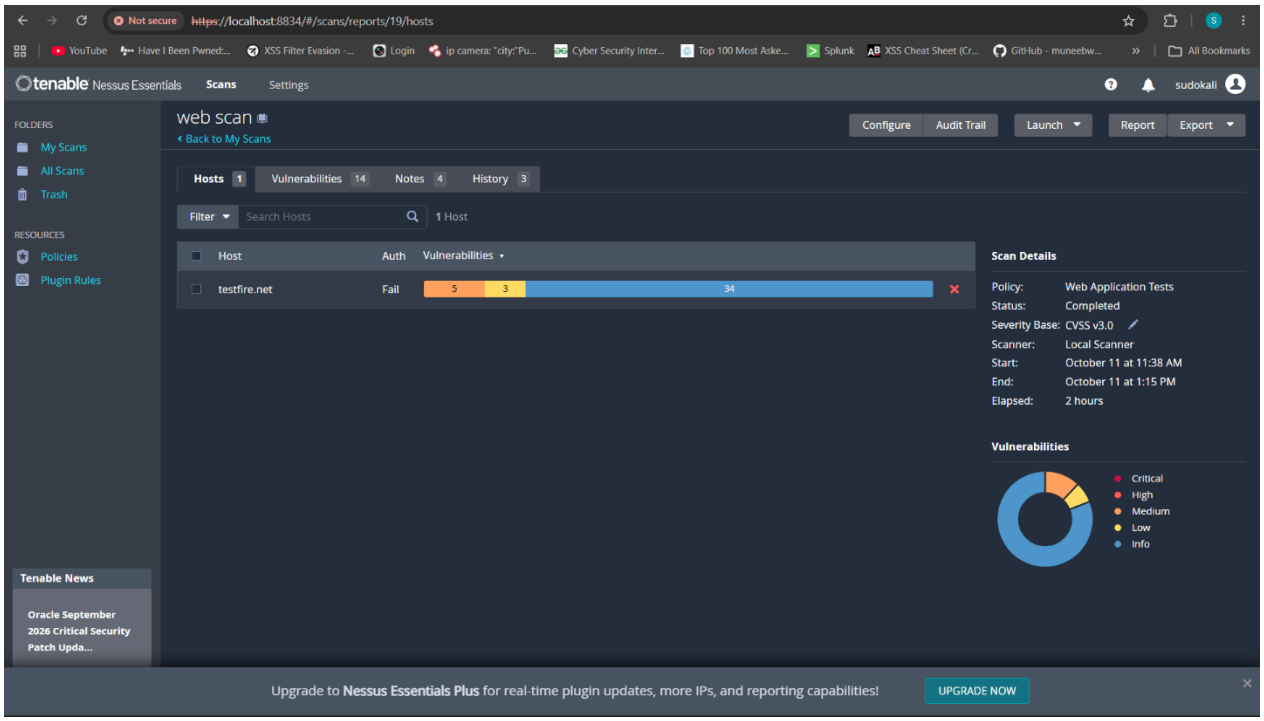
Nikto Scan

- **nikto -h http://target.com**

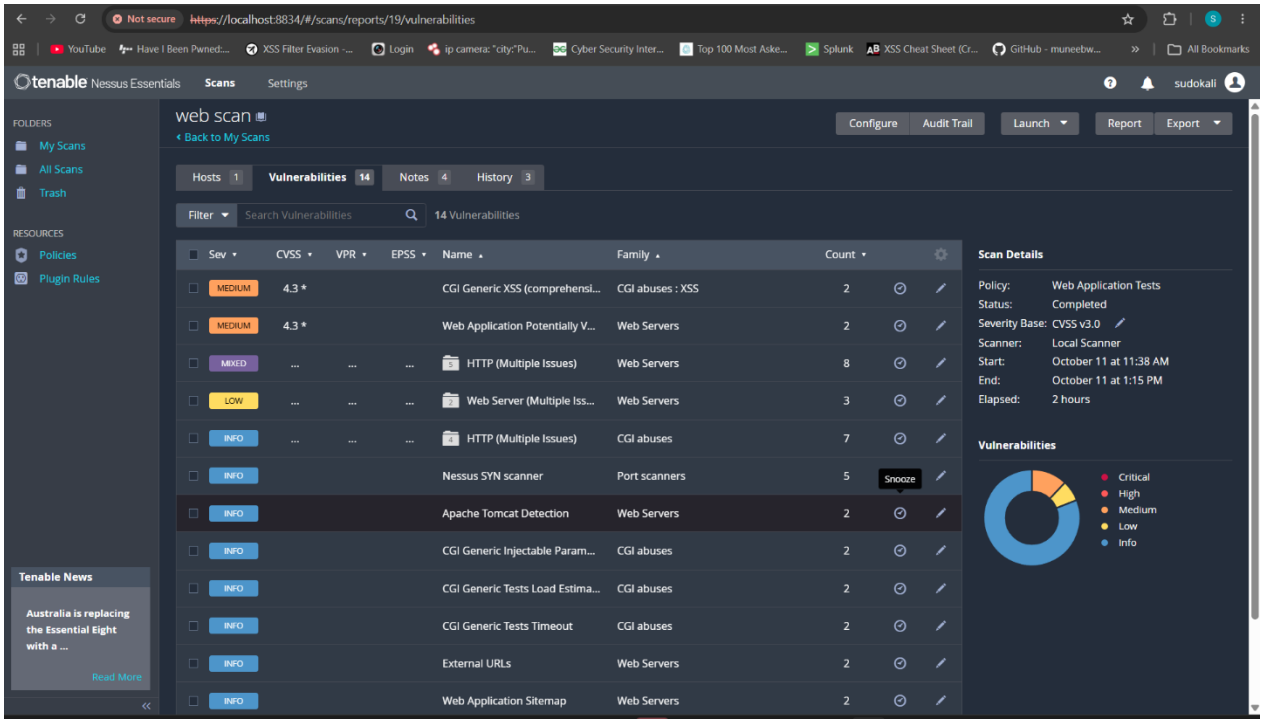
Phase 5: Vulnerability Assessment

In addition to Nikto (Phase 5), automated vulnerability scans were also performed using Nessus Essentials and Acunetix to cross-verify findings across multiple scanning engines.

Nessus Essentials Scan

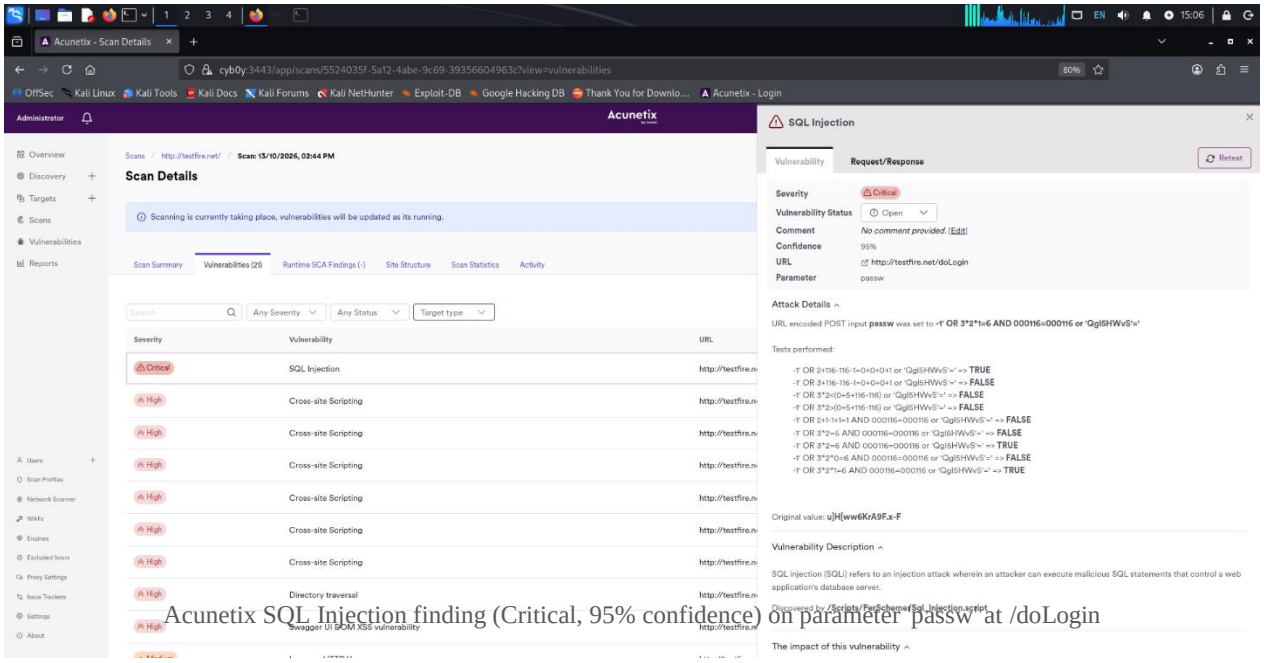


Nessus scan summary for testfire.net — 5 Medium, 3 Low, 34 Informational findings

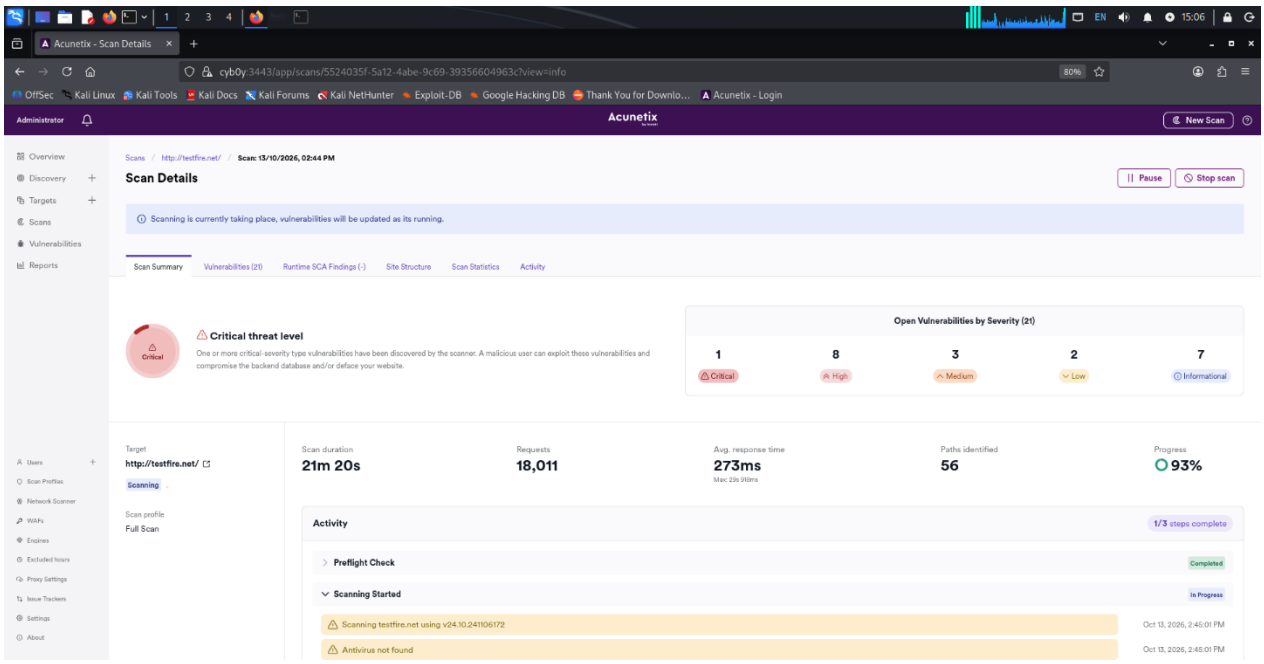


Nessus vulnerability list — CGI Generic XSS, Web Application Potentially Vulnerable, HTTP (Multiple Issues)

Phase 5: Vulnerability Assessment Acunetix Scan



Acunetix SQL Injection finding (Critical, 95% confidence) on parameter 'passw' at /doLogin



Acunetix scan summary — Critical threat level: 1 Critical, 8 High, 3 Medium, 2 Low, 7 Informational

Additional Tools Used

- Nessus Essentials
- Acunetix

```
┌──(kali㉿kali)-[~]
└─$ nikto -h https://uptoskills.com
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Multiple IPs found: 104.21.77.200, 172.67.211.113, 2606:4700:3033::6815:4dc8, 2606:4700:3035::ac43:d371
+ Target IP:          104.21.77.200
+ Target Hostname:    uptoskills.com
+ Target Port:        443
---------------------------------------------------------------------------
+ SSL Info:        Subject:  /CN=uptoskills.com
                   Ciphers:  TLS_AES_256_GCM_SHA384
                   Issuer:   /C=US/O=Google Trust Services/CN=WE1
+ Start Time:         2026-02-28 06:15:43 (GMT-5)
---------------------------------------------------------------------------
+ Server: cloudflare
+ /: Retrieved access-control-allow-origin header: *.
+ /: Uncommon header 'server-timing' found, with multiple values: (cfCacheStatus;desc="DYNAMIC",cfEdge;dur=6,cfOrigin;du
+ /: Uncommon header 'speculation-rules' found, with contents: "/cdn-cgi/speculation".
+ /: An alt-svc header was found which is advertising HTTP/3. The endpoint is: ':443'. Nikto cannot test HTTP/3 over QUI
lla.org/en-US/docs/Web/HTTP/Headers/alt-svc
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /robots.txt: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of th
to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ /robots.txt: contains 23 entries which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossa
+ /: The Content-Encoding header is set to "deflate" which may mean that the server is vulnerable to the BREACH attack.
+ ERROR: Error limit (20) reached for host, giving up. Last error: opening stream: can't connect: SSL negotiation failed
:ssl/tls alert handshake failure at /var/lib/nikto/plugins/LW2.pm line 5254.
 at /var/lib/nikto/plugins/LW2.pm line 5254.
;  at /var/lib/nikto/plugins/LW2.pm line 5254.
+ Scan terminated: 20 error(s) and 7 item(s) reported on remote host
+ End Time:           2026-02-28 06:17:28 (GMT-5) (105 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Vulnerabilities Detected

- Missing Security Headers
- Server Version Disclosure
- Directory Listing Enabled
- Outdated Software
- Weak SSL Configuration

Phase 6: Manual Penetration Testing

Automated scanners cannot identify every vulnerability. Therefore, manual testing is performed.

6.1 Authentication Testing

Objective

Evaluate authentication mechanisms.

Tests Performed

- Weak Password Testing
- Default Credentials
- Password Complexity
- Password Reset
- Account Lockout
- Multi-factor Authentication

Expected Result

Only authorized users should successfully log in.

6.2 SQL Injection Testing

Objective

Determine whether user inputs interact securely with the database.

Tools

- SQLMap
- Burp Suite

Example SQL Injection Payload

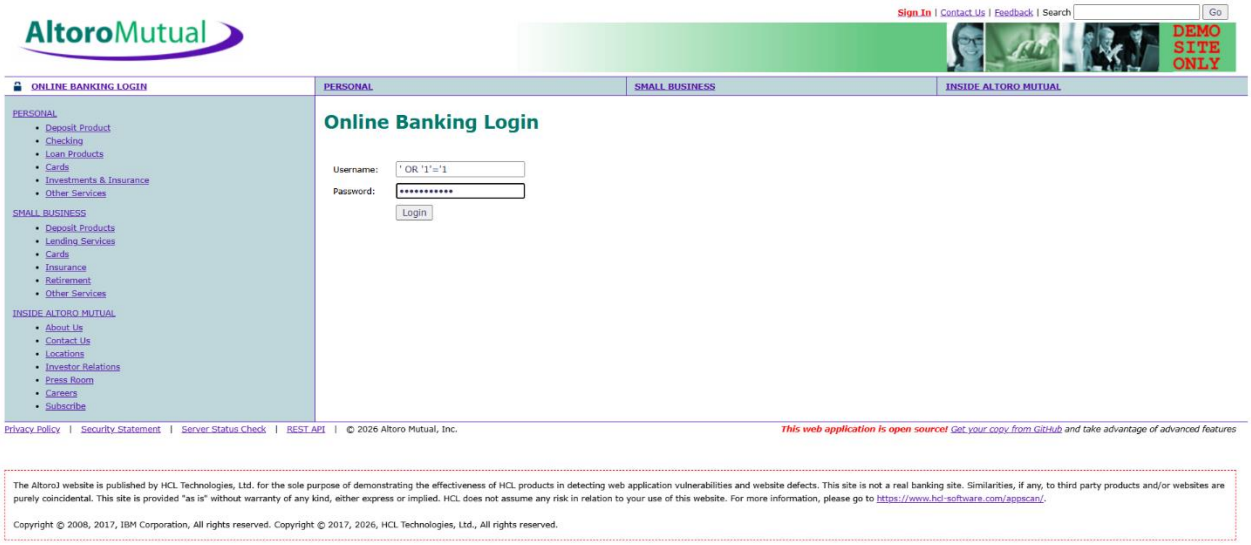
' OR '1'='1

SQLMap Command

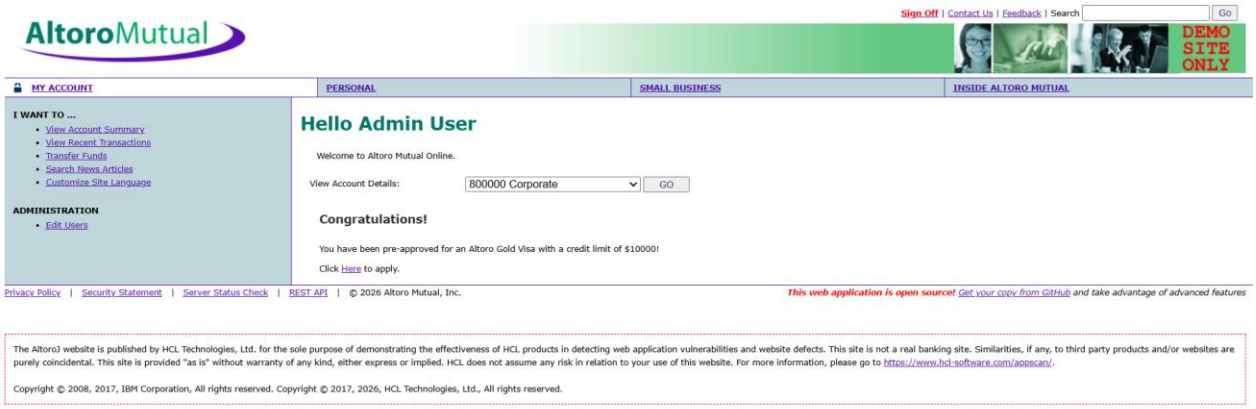
- **sqlmap -u "http://target.com/product.php?id=1" –dbs**

6.2 SQL Injection Testing

Manual testing was performed on the Online Banking Login form (Section 6.2, SQL Injection Testing) using classic authentication-bypass payloads, and the request/response traffic was captured and analyzed using Burp Suite Professional.

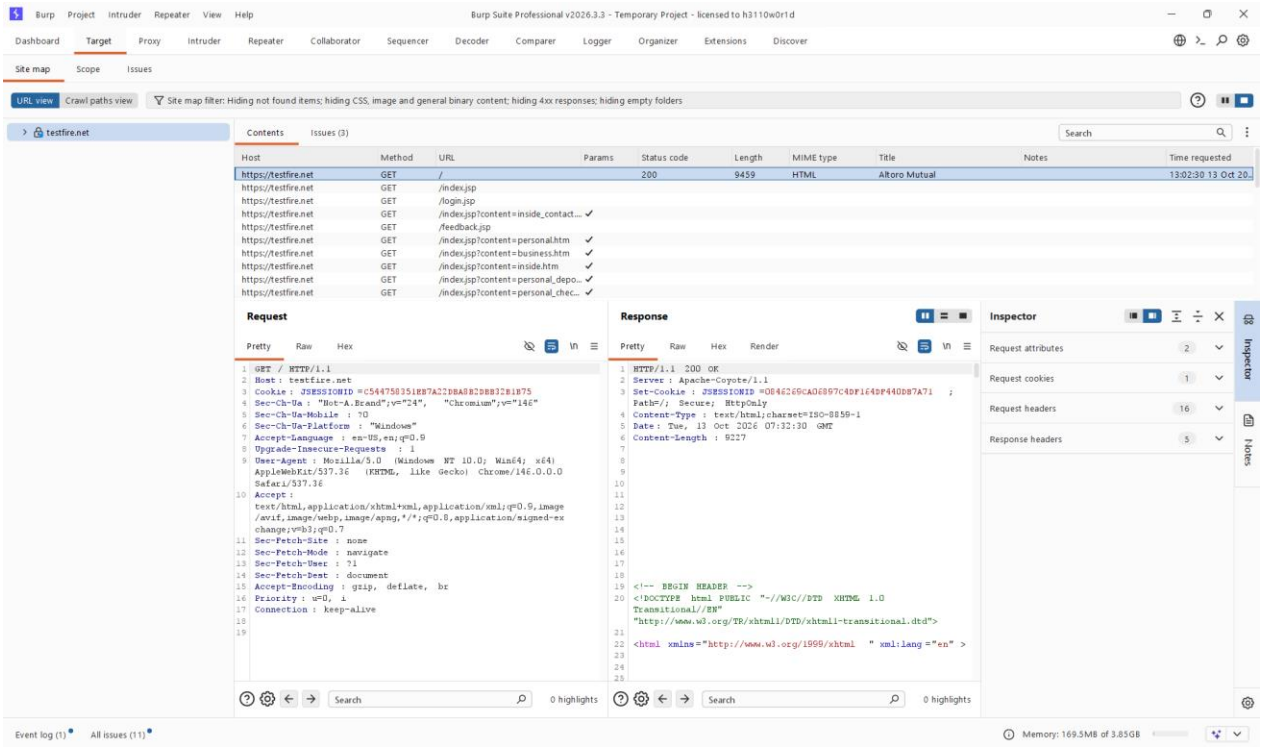


Login form submitted with payload ' OR '1'='1 in the Username field

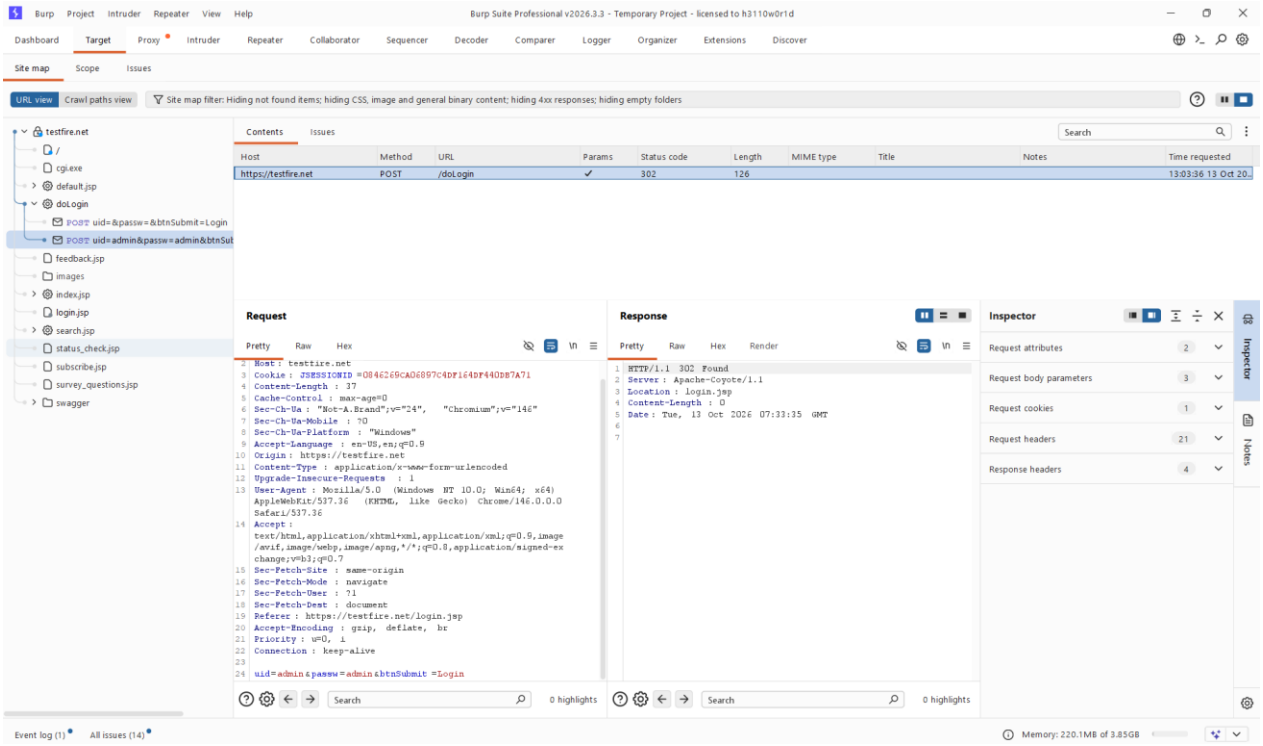


Authentication bypass successful — logged in as Admin User without valid credentials

6.2 SQL Injection Testing — Burp Suite Request Capture



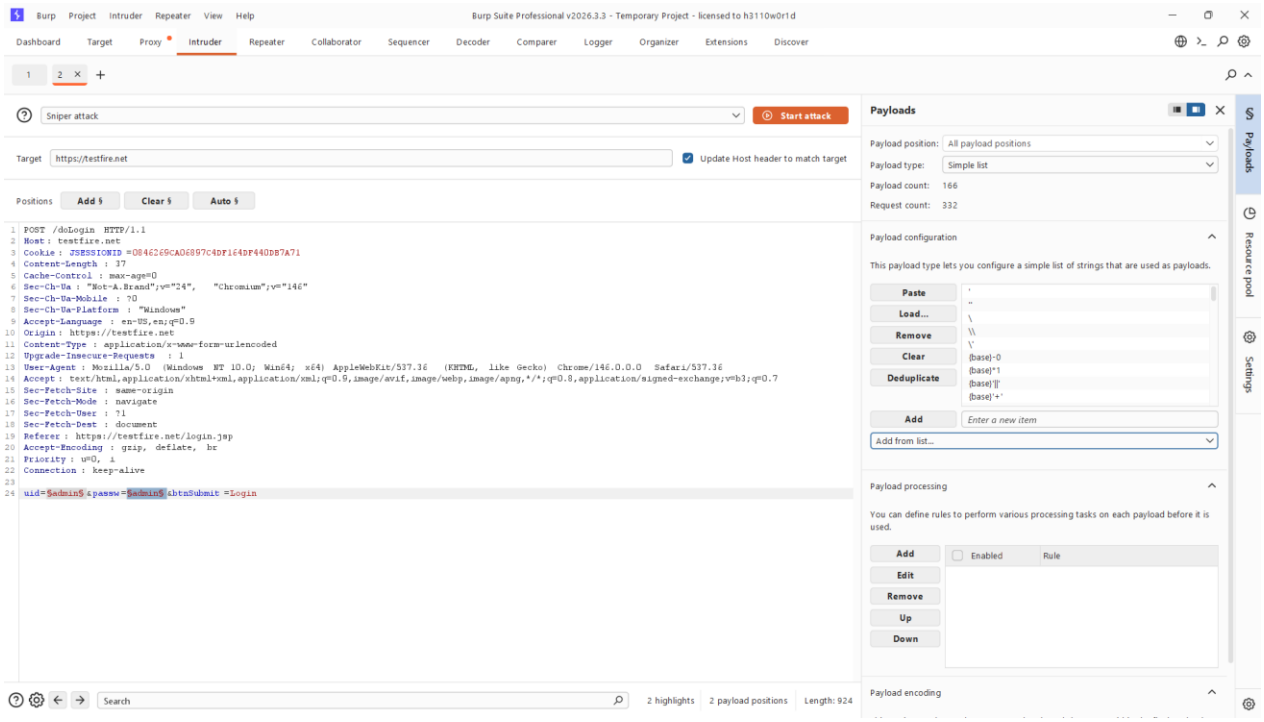
Burp Suite Target site map — application crawled and requests captured



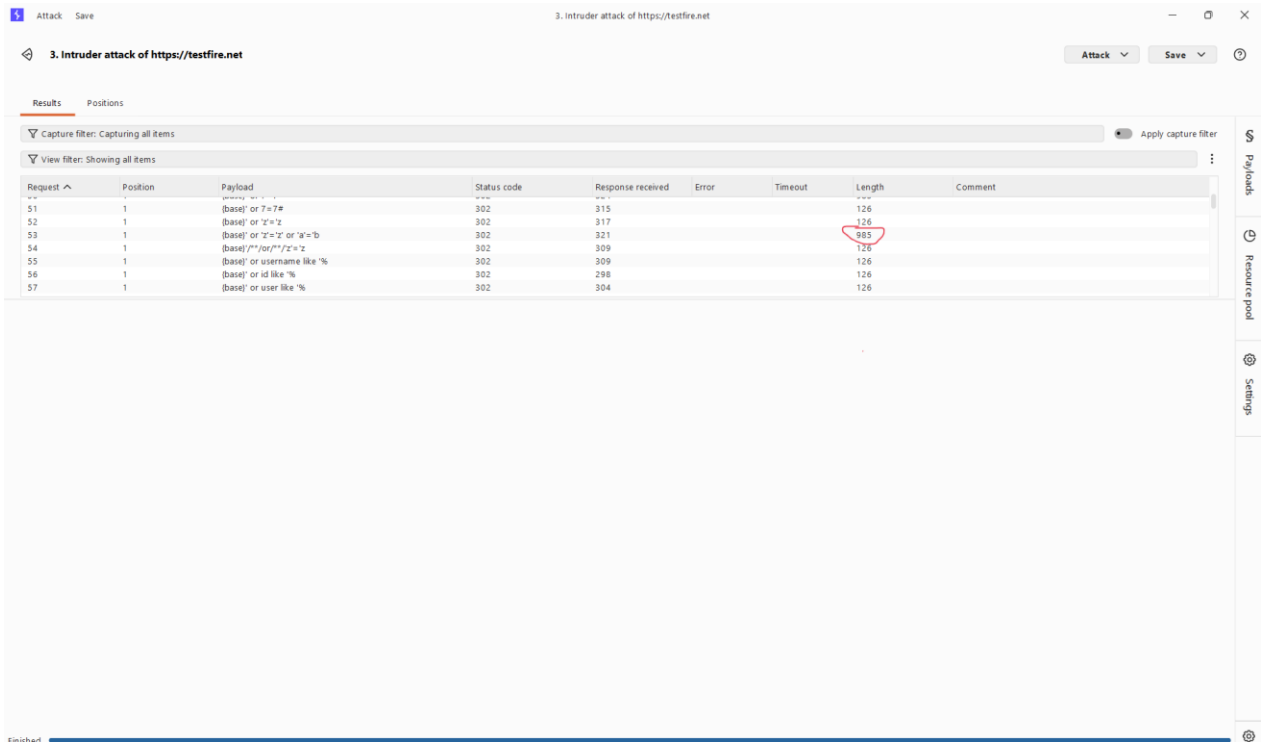
Burp Suite capture of POST /doLogin request — uid=admin&passw=admin

6.2 SQL Injection Testing — Burp Intruder Fuzzing

Burp Intruder was used to automate testing of multiple SQL injection payload variations against the login parameters to identify anomalous responses.



Burp Intruder — Sniper attack configured with payload positions on uid and passw parameters



Burp Intruder results — anomalous response length (985) flags a potential injection point

Cross-Site Scripting (XSS) Testing

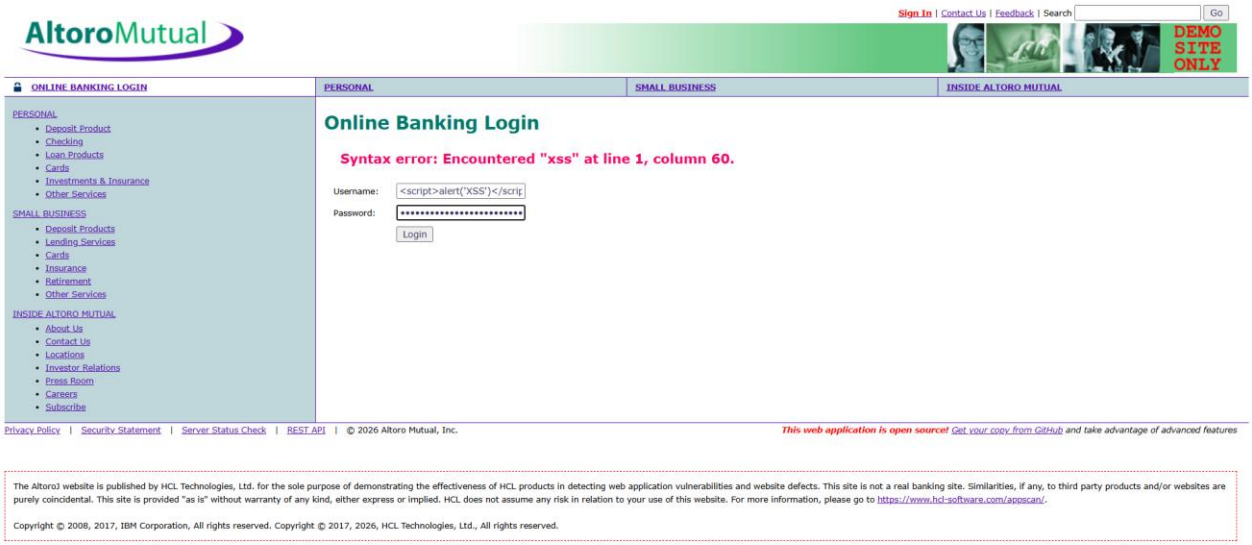
Objective: Determine whether user-supplied input is properly sanitized and encoded before being reflected back by the application.

Tools

- Manual Testing (Browser)
- Burp Suite Intruder

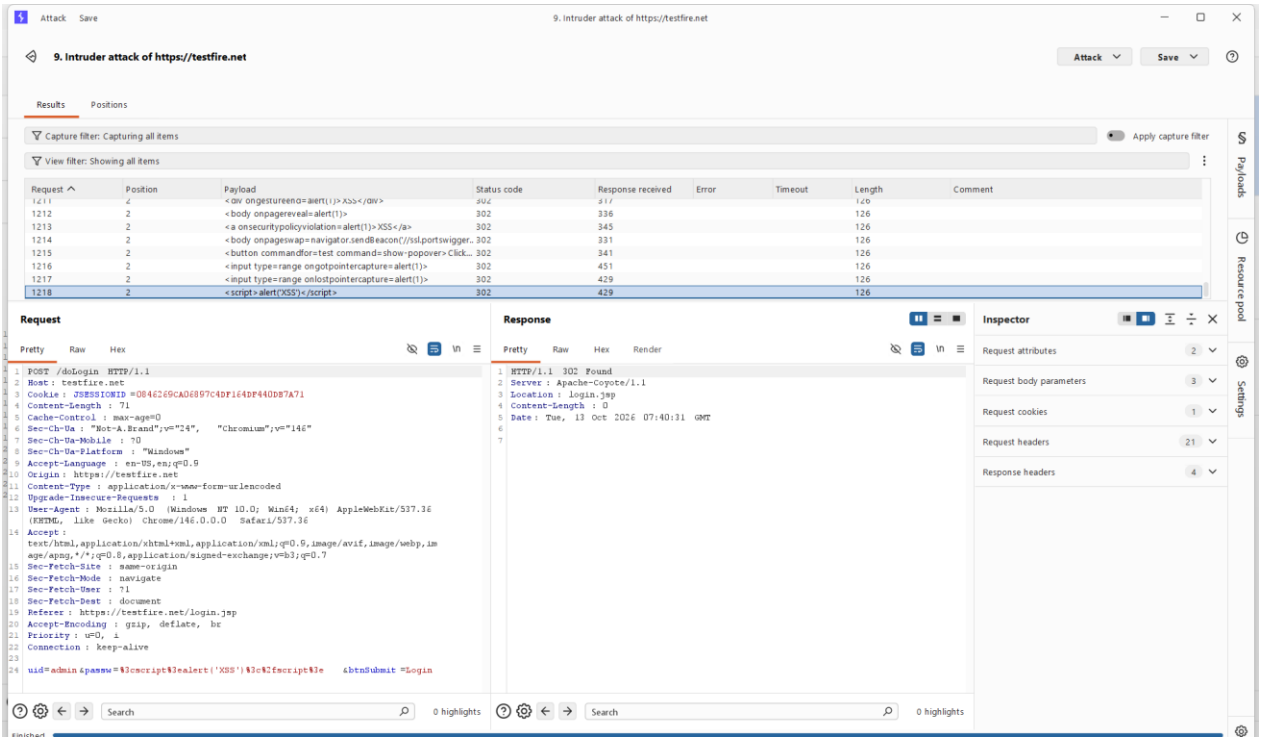
Example XSS Payload

<script>alert('XSS')</script>



XSS payload submitted in the login form — application returned a parser syntax error

Cross-Site Scripting (XSS) Testing



Burp Intruder XSS payload fuzzing — multiple event-handler and script-tag payloads tested

Risk

Medium — reflected input is echoed back by the application without proper encoding, indicating a potential Cross-Site Scripting vulnerability.

Recommendation

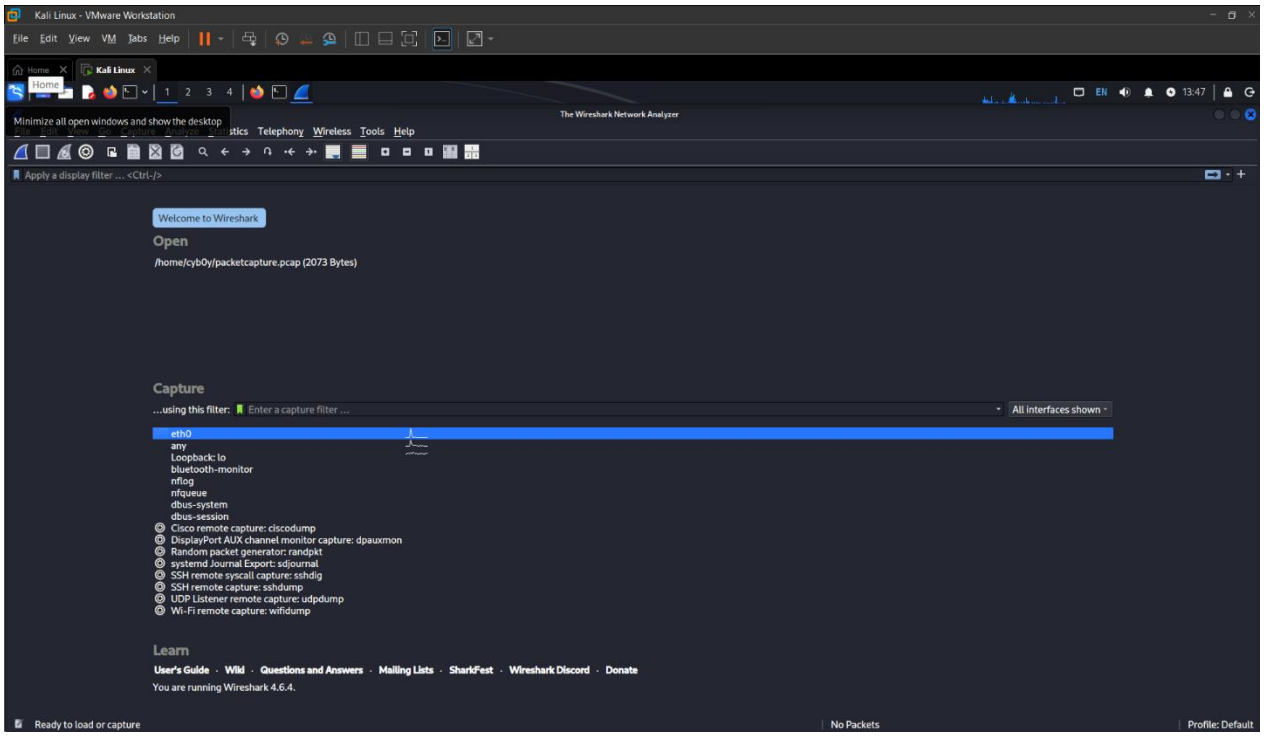
- Encode all output.
- Sanitize user input.
- Deploy a Content Security Policy.

Network Traffic Analysis (Wireshark)

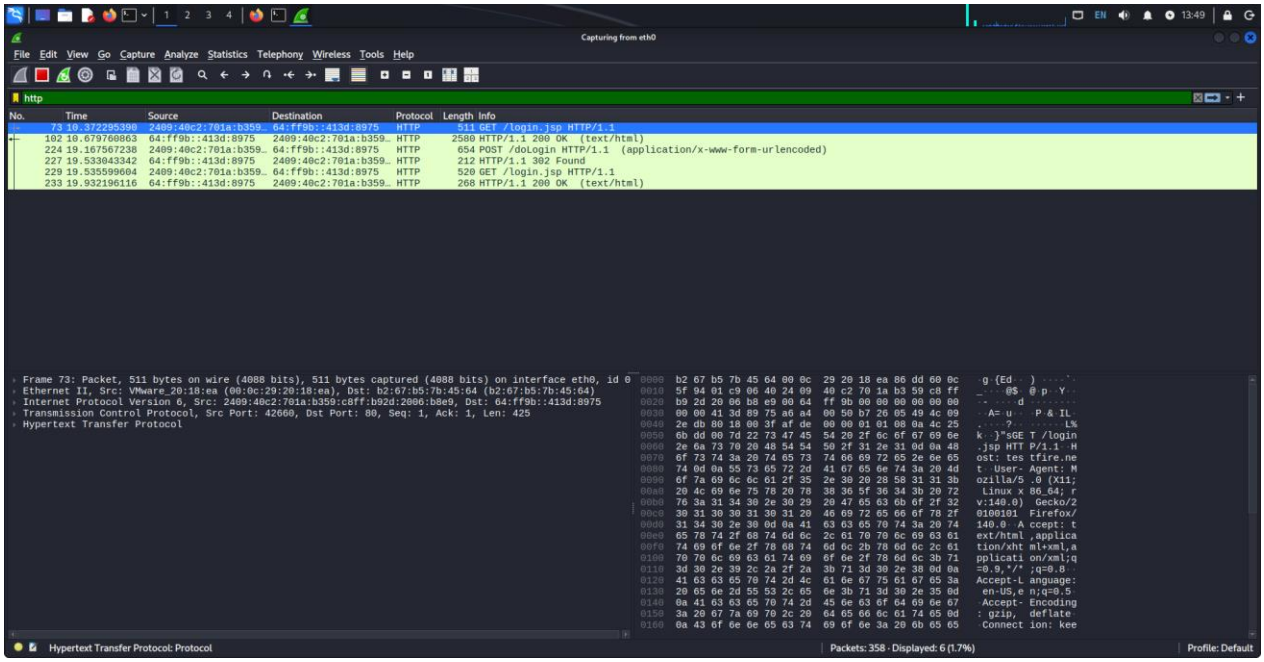
Objective: Capture and analyze live network traffic between the client and the web server to identify any sensitive data transmitted without encryption.

Tool

- Wireshark

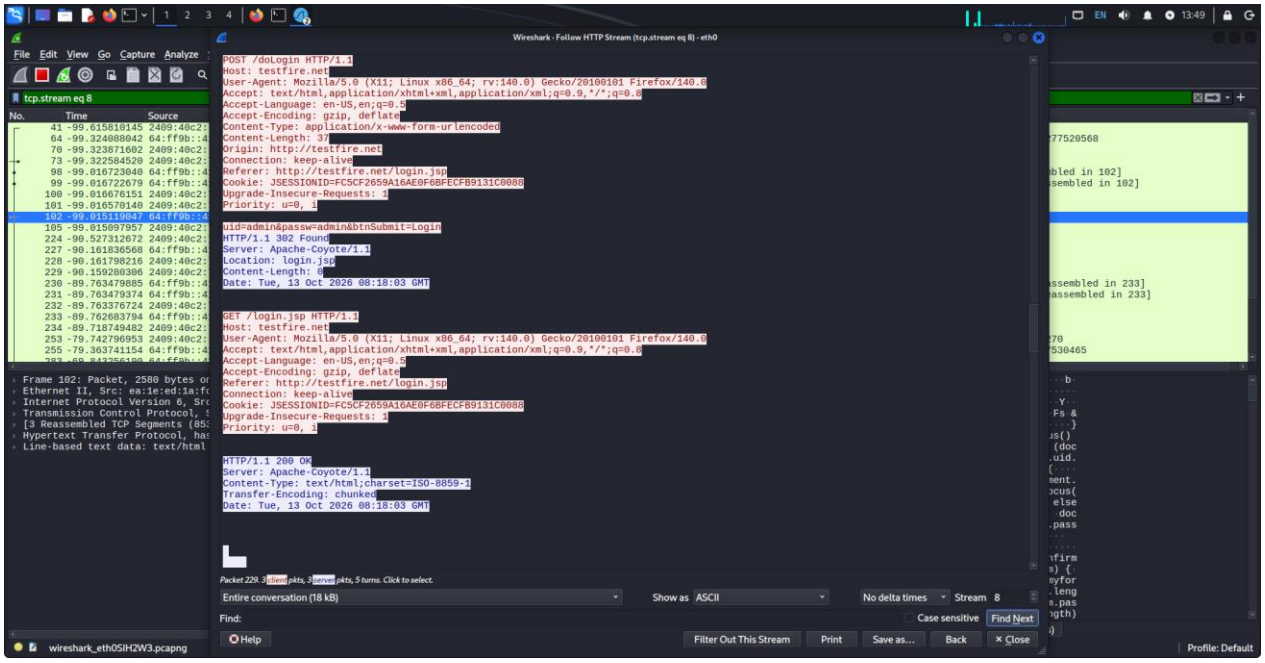


Wireshark packet capture started on interface eth0 (Kali Linux)



HTTP filter applied — login requests GET /login.jsp and POST /doLogin captured

Network Traffic Analysis (Wireshark) (continued)



Follow HTTP Stream — login credentials uid=admin&passw=admin captured in plaintext

Finding

Login credentials were transmitted over unencrypted HTTP and were fully readable in the captured packet stream, exposing them to network-level interception.

Risk

High — Sensitive Data Exposure (credentials transmitted in cleartext).

Recommendation

- Enforce HTTPS/TLS for all authentication traffic.
- Enable HTTP Strict Transport Security (HSTS).
- Disable plaintext HTTP access to login endpoints.



Risk

Critical

Impact

- Database Dump
- Credential Theft
- Data Manipulation
- Complete Database Access

Recommendation

- Use Prepared Statements
- Parameterized Queries
- Input Validation

6.3 SSL/TLS Testing

Tools

- SSL Labs

- testssl.sh

Tests

- HTTPS Enabled
- TLS Version
- Weak Cipher Suites
- Certificate Validity

Phase 7: Exploitation

After identifying vulnerabilities, controlled exploitation is performed to verify their existence.

Examples

- Successfully retrieve database names using SQL Injection.
- Execute JavaScript alert using XSS.
- Access another user's profile through IDOR.
- Upload a test file to demonstrate insecure file upload.

Evidence such as screenshots and HTTP requests/responses should be collected during this phase.

Phase 8: Risk Assessment

Each vulnerability is assigned a severity based on the Common Vulnerability Scoring System (CVSS).

Vulnerability	Severity (CVSS Score)	Risk Level
SQL Injection	9.8	Critical
Stored XSS	8.8	High
Broken Authentication	8.1	High
IDOR	7.5	High
File Upload	8.6	High
Missing Security Headers	4.3	Medium
Directory Listing	3.7	Low

Phase 9: Remediation

SQL Injection

- Use Prepared Statements.
- Implement Parameterized Queries.
- Validate all user inputs.

Cross-Site Scripting

- Encode all output.
- Sanitize user input.
- Deploy a Content Security Policy.

Broken Authentication

- Enforce strong passwords.
- Enable Multi-Factor Authentication.
- Secure session management.

File Upload

- Restrict allowed file types.
- Verify MIME types.
- Scan uploaded files with antivirus.
- Store uploads outside the web root.

Security Headers

Configure the following headers:

- Content-Security-Policy
- Strict-Transport-Security
- X-Frame-Options
- X-Content-Type-Options
- Referrer-Policy

Tools Used

Tool	Purpose
Nmap	Network Scanning
WhatWeb	Technology Detection
Gobuster	Directory Enumeration
Burp Suite	Intercepting and Manual Testing
OWASP ZAP	Automated Web Vulnerability Scanning
Nikto	Web Server Vulnerability Scanning
SQLMap	SQL Injection Testing
Nuclei	Template-Based Vulnerability Scanning
Wireshark	Network Packet Analysis
Firefox + FoxyProxy	Proxy Configuration

Conclusion:

The VAPT assessment successfully identified multiple security vulnerabilities affecting the confidentiality, integrity, and availability of the web application. Automated scanning combined with manual penetration testing provided a comprehensive evaluation of the application's security posture. Critical issues such as SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, and Insecure Direct Object References (IDOR) demonstrated the importance of secure coding practices, proper input validation, robust authentication mechanisms, and secure server configuration.